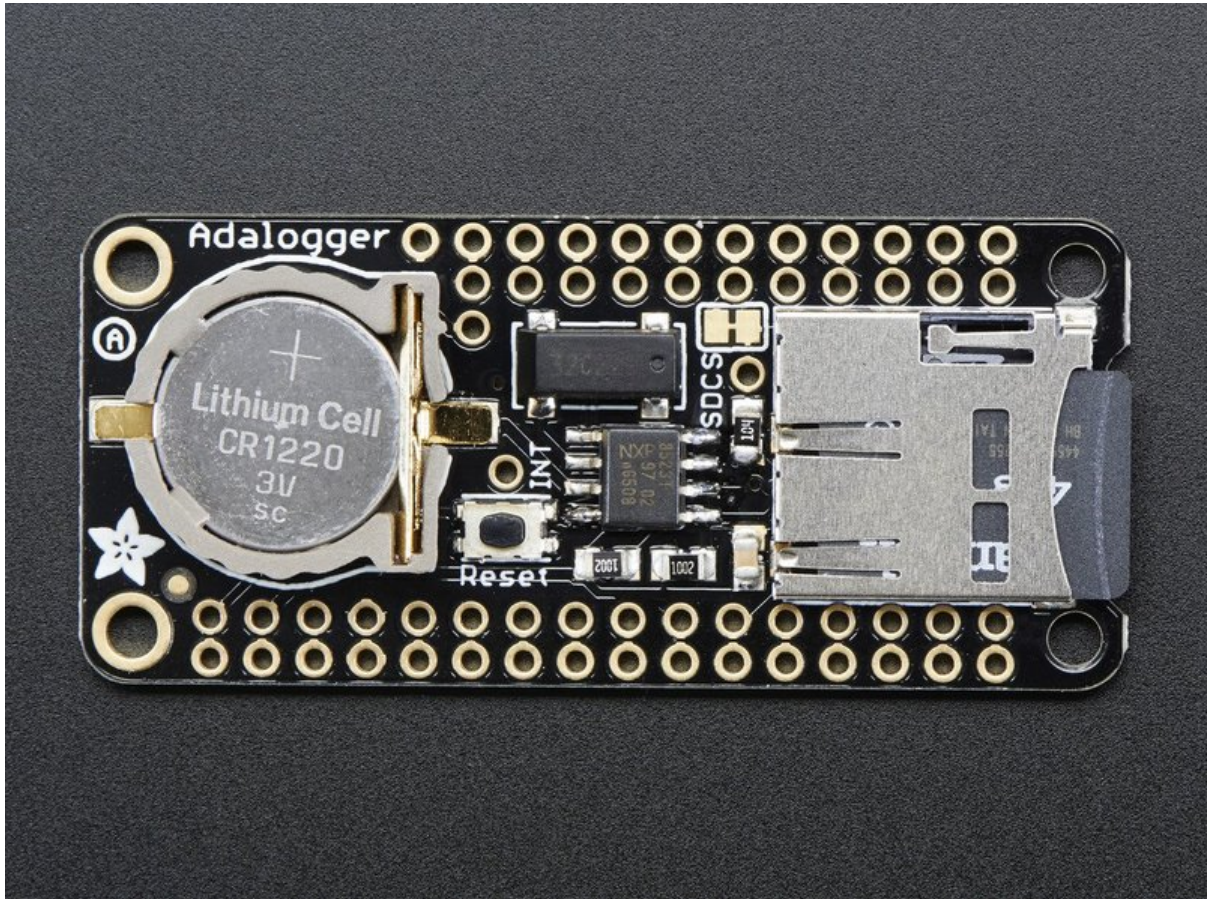




Adafruit Adalogger FeatherWing

Created by lady ada



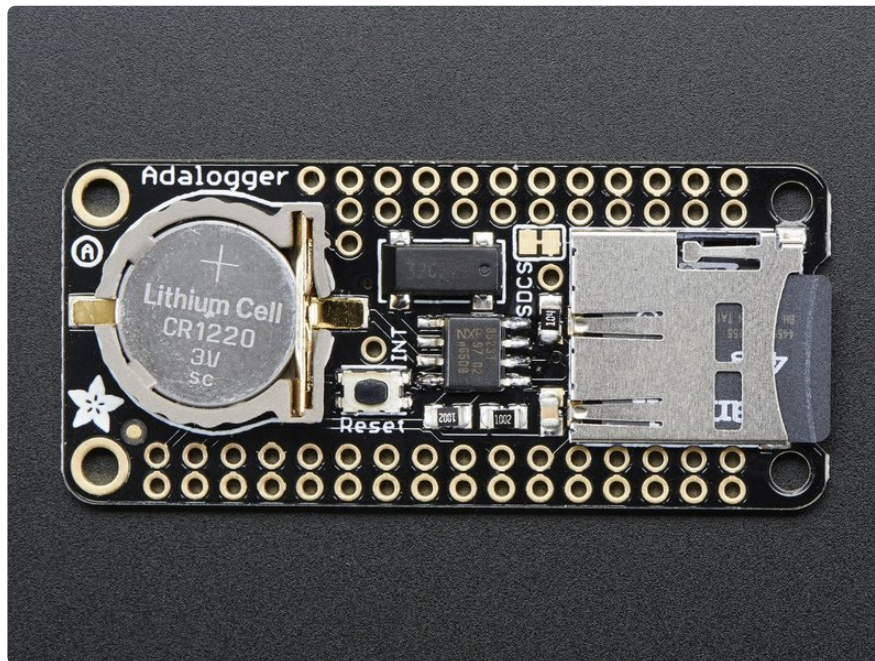
<https://learn.adafruit.com/adafruit-adalogger-featherwing>

Last updated on 2025-05-20 03:37:10 PM EDT

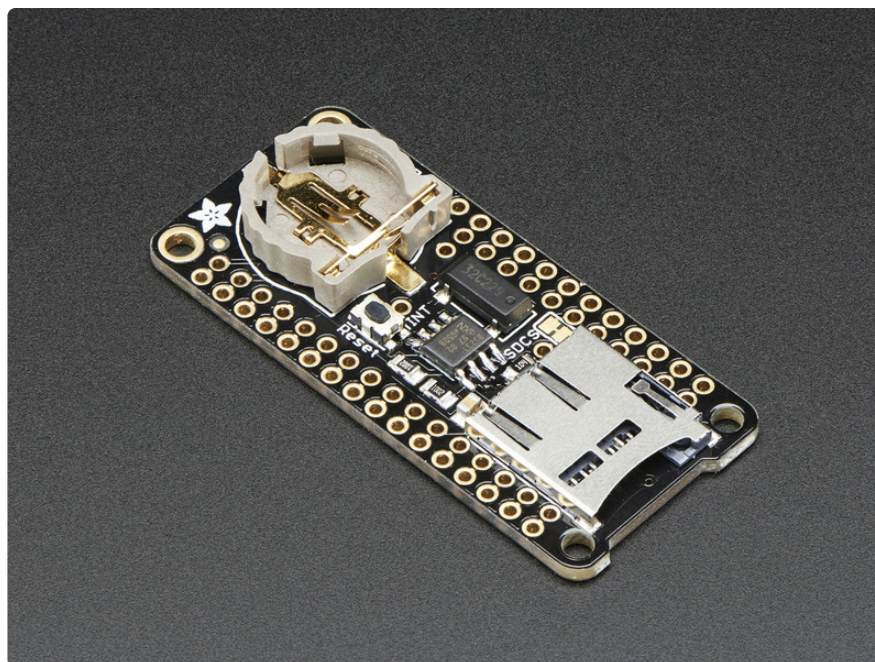
Table of Contents

Overview	3
Pinouts	5
• Power Pins • RTC & I2C Pins • SD & SPI Pins	
Assembly	8
Using the Real Time Clock	10
• What is a Real Time Clock? • Battery Backup	
RTC with Arduino	11
• Wiring • Talking to the RTC • First RTC test • Setting the time • Reading the time	
RTC with CircuitPython	17
• Wiring • Adafruit CircuitPython Library Install • Usage • Setting the time	
Using the SD Card	21
• Formatting under Windows/Mac • Basic SD Card Test • Next steps! • Example logging sketch	
CircuitPython	28
• Adafruit CircuitPython Module Install • Usage • Initialize & Mount SD Card Filesystem Using sdcardio • Initialize & Mount SD Card Filesystem Using adafruit_sdcard • Reading & Writing Data • List Files • Logging Temperature	
Downloads	38
• Datasheets and Files • Schematic • Fabrication Print	

Overview

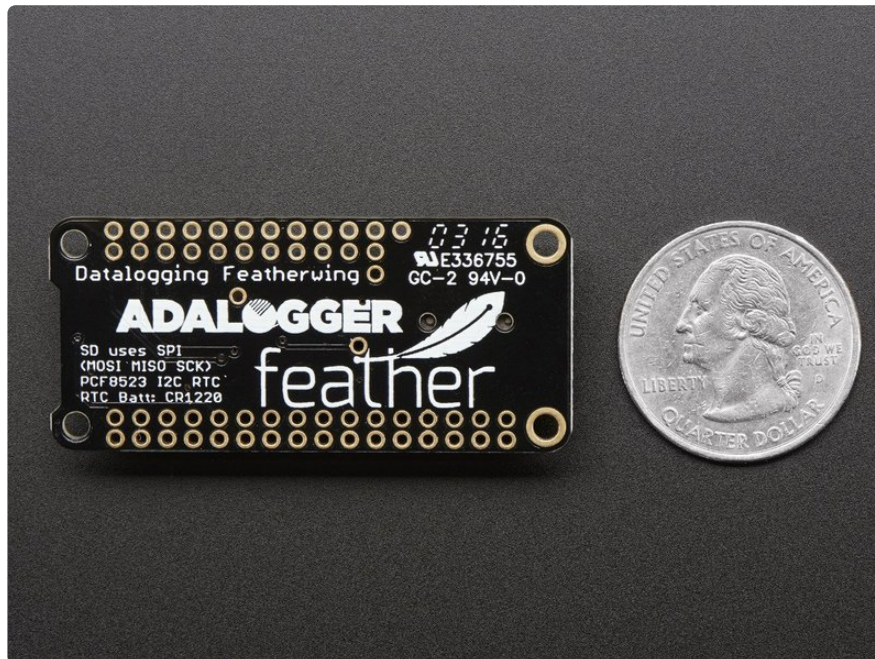


A Feather board without ambition is a Feather board without FeatherWings! This is the **Adalogger FeatherWing**: it adds both a battery-backed Real Time Clock and micro SD card storage to any Feather main board. Using our [Feather Stacking Headers](http://adafru.it/2830) (<http://adafru.it/2830>) or [Feather Female Headers](http://adafru.it/2886) (<http://adafru.it/2886>) you can connect a FeatherWing on top of your Feather board and let the board take flight!

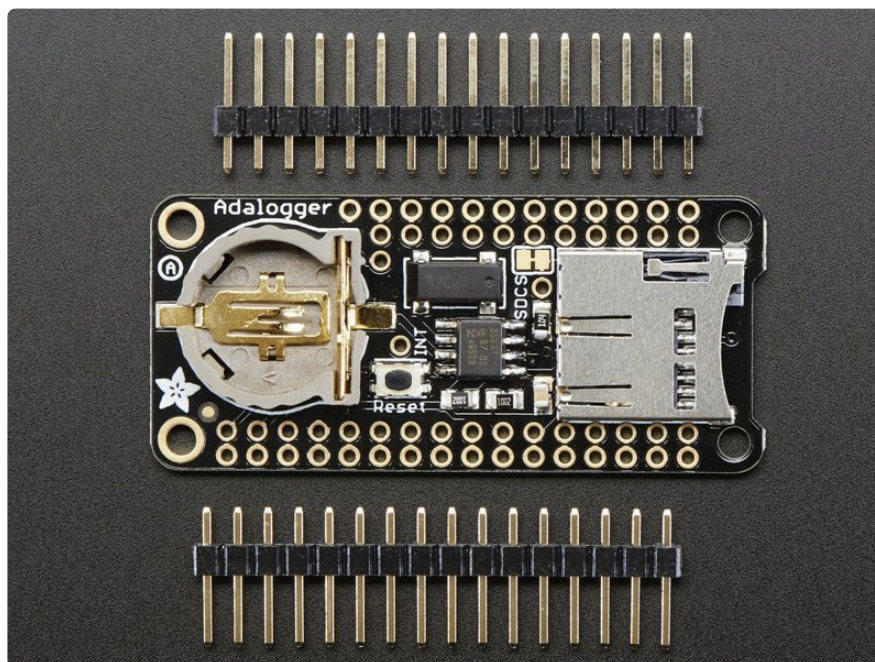


This FeatherWing will make it real easy to add datalogging to any of our existing Feathers. You get both an I2C real time clock (PCF8523) with 32KHz crystal and battery backup, and a microSD socket that connects to the SPI port pins (+ extra pin

for CS). Tested and works great with any of our Feathers, based on ATmega32u4, ATmega328P, ATSAM51, ATSAM51, nRF52, Teensy, or ESP32/ESP8266.

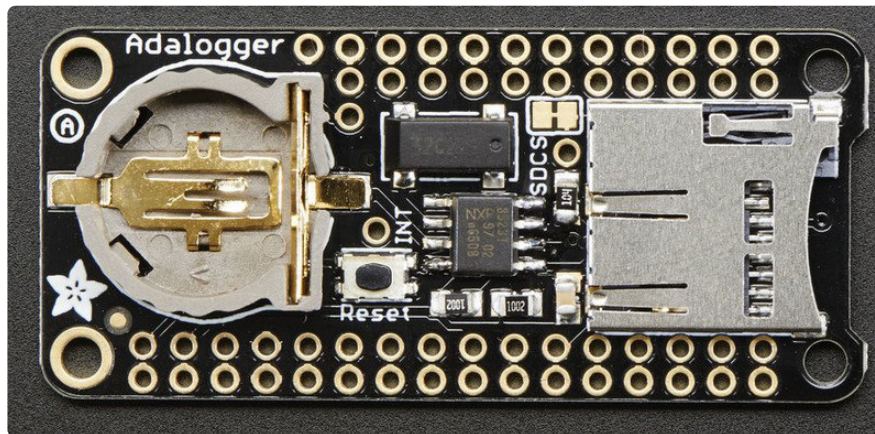


We recommend the Arduino's default SD library to talk to the microSD card socket. On ESP8266, the SD CS pin is on GPIO 15, on Atmel M0, M4, 328P or 32u4 it's on GPIO 10. You can cut the trace to the default pin and change this to any pin. To use the RTC, [use our RTCLib library \(https://adafru.it/c7r\)](https://adafru.it/c7r). If you need a [precision RTC, check out our DS3231 FeatherWing \(http://adafru.it/3028\)](http://adafru.it/3028)



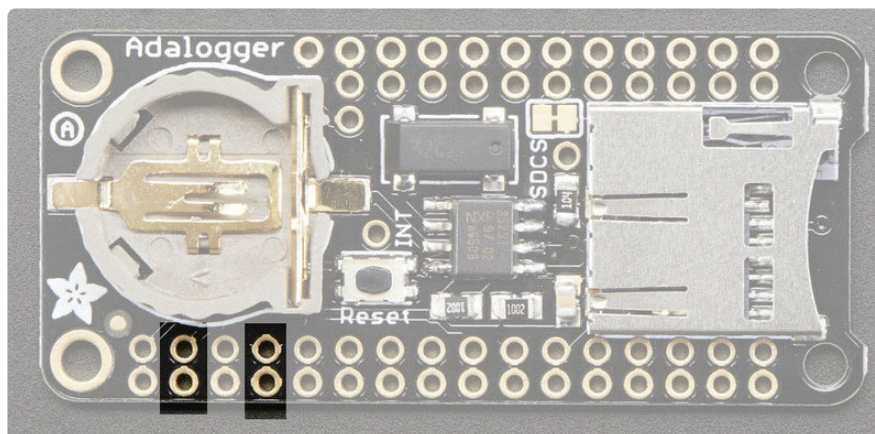
Great for any kind of datalogging or even data reading! Some light soldering is required to attach the headers onto the 'Wing but it's a 10 minute task.

Pinouts



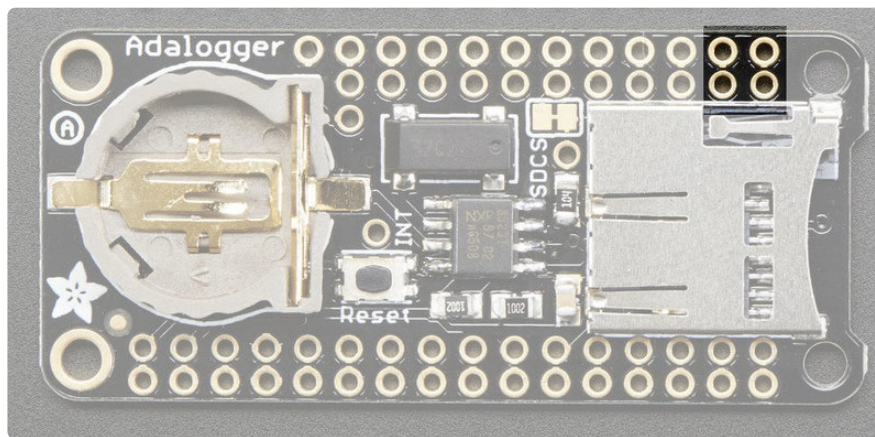
Even though every pin from the Feather is 'doubled up' with an inner header, not all of the pins are actually used!

Power Pins



On the bottom row, the **3.3V** (second from left) and **GND** (fourth from left) pin are used to power the SD card and RTC (to take a load off the coin cell battery when main power is available)

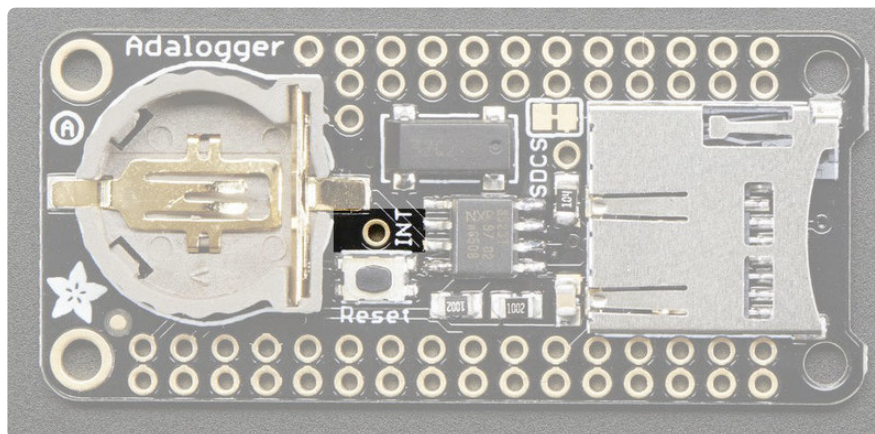
RTC & I2C Pins



In the top right, **SDA** (rightmost) and **SCL** (to the left of SDA) are used to talk to the RTC chip.

- **SCL** - I2C clock pin, connect to your microcontrollers I2C clock line. This pin has a 10K pullup resistor to 3.3V
- **SDA** - I2C data pin, connect to your microcontrollers I2C data line. This pin has a 10K pullup resistor to 3.3V

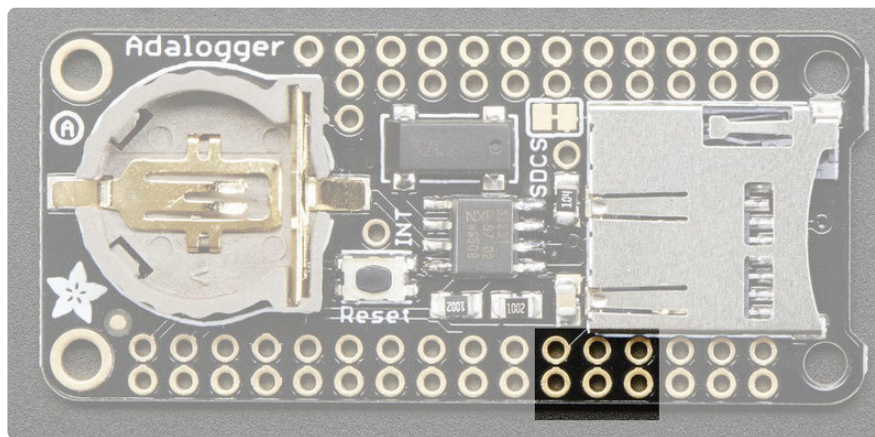
These pins are in the same location on every Feather



There's also a breakout for **INT** which is the output pin from the RTC. It can be used as an interrupt output or it could also be used to generate a square wave.

Note that this pin is **open drain** - you must enable the internal pullup on whatever digital pin it is connected to!

SD & SPI Pins



Starting from the left you've got

- **SPI Clock (SCK)** - output from feather to wing
- **SPI Microcontroller Out Sensor In (MOSI)** - output from feather to wing
- **SPI Microcontroller In Sensor Out (MISO)** - input from wing to feather

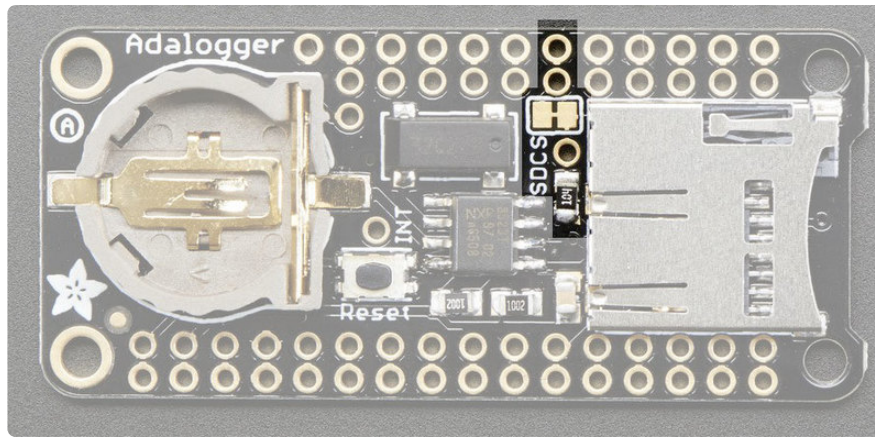
These pins are in the same location on every Feather. They are used for communicating with the SD card. When the SD card is not inserted, these pins are completely free. MISO is tri-stated whenever the **SD CS** pin is pulled high

The **SDCS** pin is the chip select line.

- On ESP8266, the SD CS pin is on GPIO **15**
- On ESP32 it's GPIO **33**
- On WICED it's GPIO **PB5**
- On the nRF52832 it's GPIO **11**
- On Atmel M0, M4, 328p or 32u4 it's on GPIO **10**
- On Teensy 3.x it's on GPIO **10**

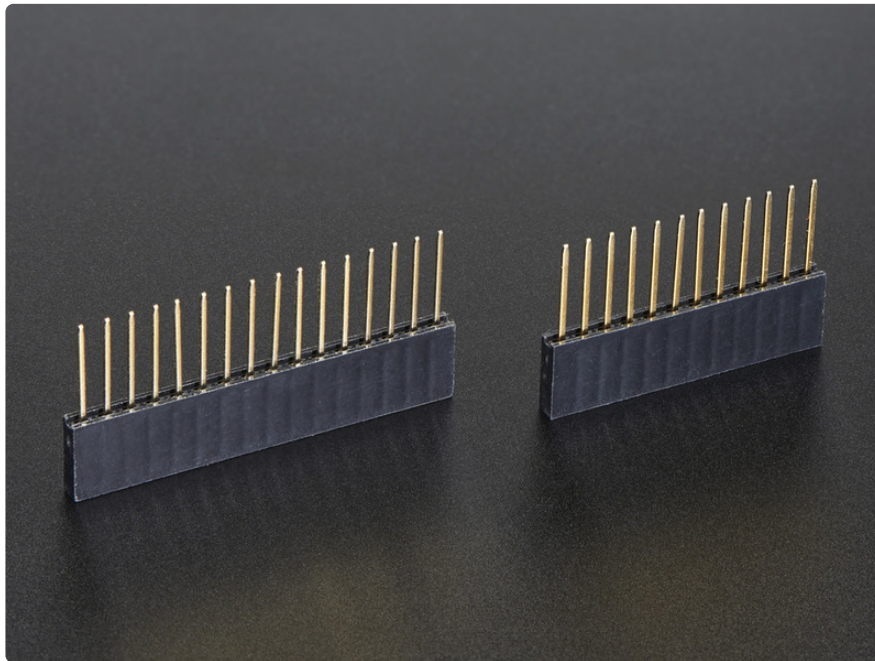
Some users report ESP32 variants may use GPIO10 instead. For the FeatherS3, users report GPIO3.

You can cut the trace to the default pin and change this to any pin by soldering a wire to any available pad.

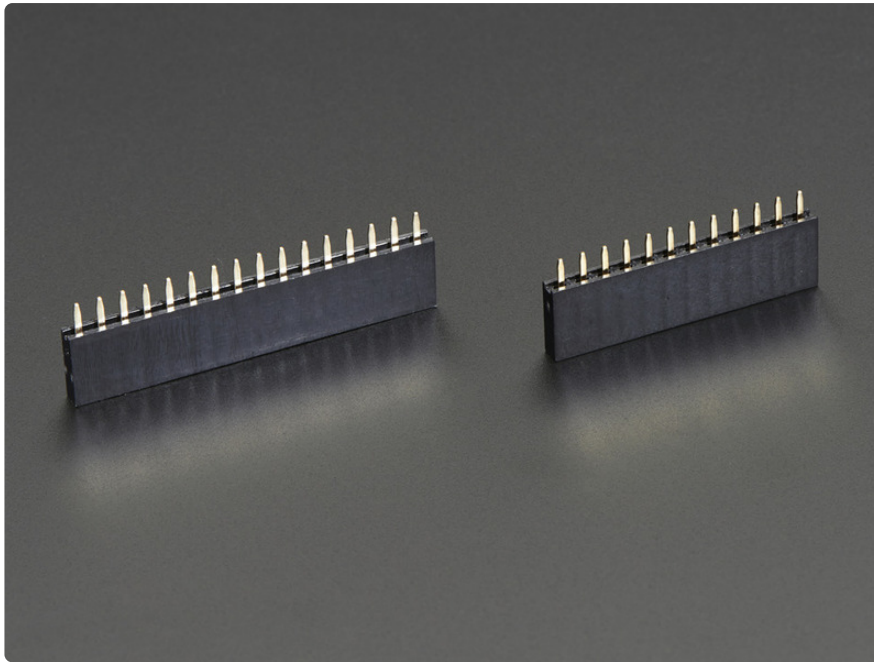


Assembly

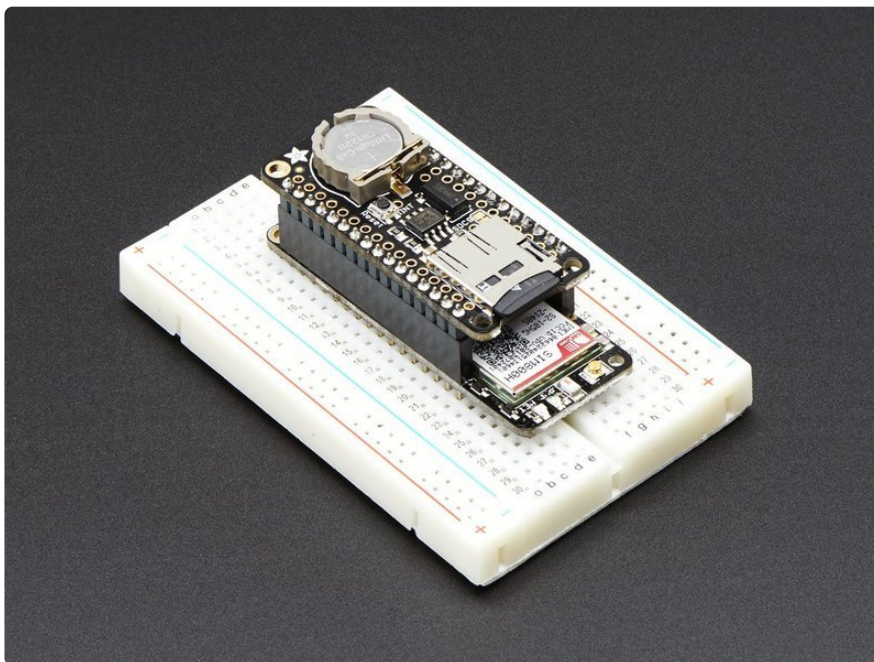
When putting together your Featherwings, think about how you want it to connect, you can use stacking headers:



Or plain female socket headers:



The most common method of attachment for the featherwing is putting stacking or female headers on the Feather mainboard and then putting the Wing on top:



But don't forget, you can **also** put the stacking headers on the wing and stack the Feather on top of it!

Using the Real Time Clock

What is a Real Time Clock?

When logging data, it's often really really useful to have timestamps! That way you can take data one minute apart (by checking the clock) or noting at what time of day the data was logged.

The Arduino IDE does have a built-in timekeeper called **millis()** and there's also timers built into the chip that can keep track of longer time periods like minutes or days. So why would you want to have a separate RTC chip? Well, the biggest reason is that **millis()** only keeps track of time since the Feather was last powered - that means that when the power is turned on, the millisecond timer is set back to 0. The Feather doesn't know it's 'Tuesday' or 'March 8th' all it can tell is 'It's been 14,000 milliseconds since I was last turned on'.

OK so what if you wanted to set the time? You'd have to program in the date and time and you could have it count from that point on. But if it lost power, you'd have to reset the time. Much like very cheap alarm clocks: every time they lose power they blink **12:00**

While this sort of basic timekeeping is OK for some projects, a data-logger will need to have **consistent timekeeping that doesn't reset when the power goes out or is reprogrammed**. Thus, we include a separate RTC! The RTC chip is a specialized chip that just keeps track of time. It can count leap-years and knows how many days are in a month, but it doesn't take care of Daylight Savings Time (because it changes from place to place)



This image shows a computer motherboard with a Real Time Clock called the [DS1387](https://adafru.it/aX0) (<https://adafru.it/aX0>). There's a lithium battery in there which is why it's so big.

The RTC we'll be using is the [PCF8523](https://adafru.it/reb) (<https://adafru.it/reb>)

Battery Backup

As long as it has a coin cell to run it, the RTC will merrily tick along for a long time, even when the Feather loses power, or is reprogrammed.

Use any CR1220 3V lithium metal coin cell battery:

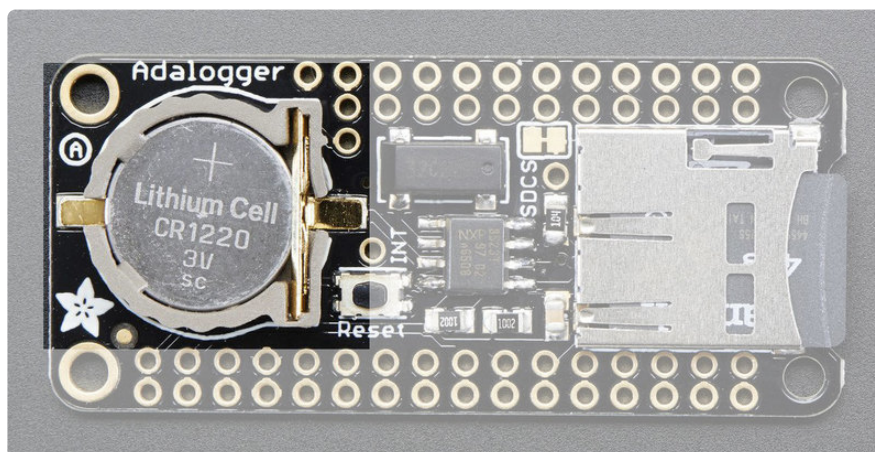


CR1220 12mm Diameter - 3V Lithium Coin Cell Battery

These are the highest quality & capacity batteries, the same as shipped with the iCufflinks, iNecklace, Datalogging and GPS Shields, GPS HAT, etc. One battery per order...

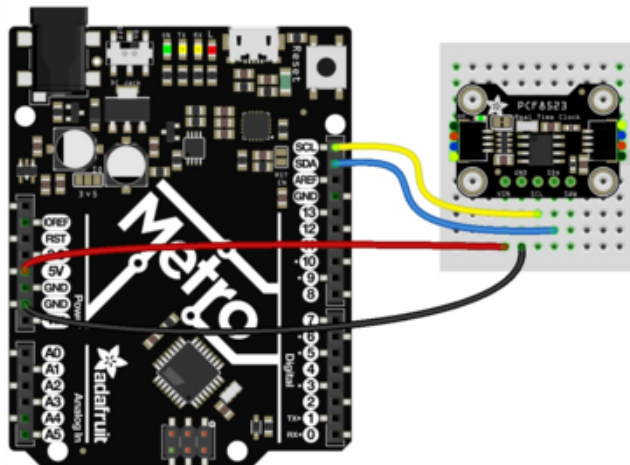
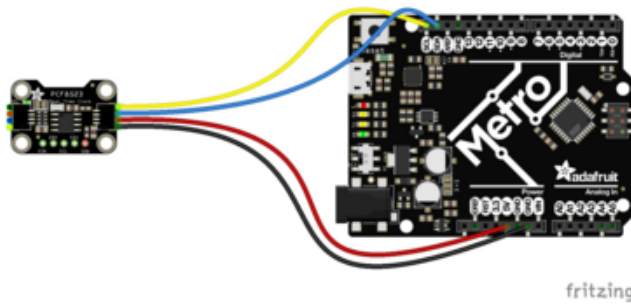
<https://www.adafruit.com/product/380>

You MUST have a coin cell installed for the RTC to work, if there is no coin cell, it will act strangely and possibly hang the Arduino when you try to use it, so ALWAYS make SURE there's a battery installed, even if it's a dead battery.



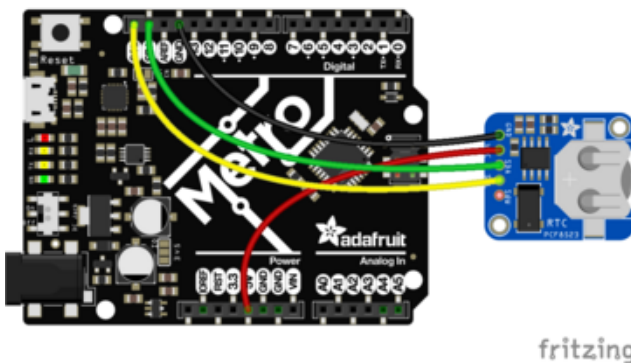
RTC with Arduino Wiring

Wiring it up is easy, connect it up as shown below.



GND to GND (black wire on **STEMMA QT** version) on your board
VCC (red wire on **STEMMA QT** version) to the logic level power of your board (on classic Arduinos & Metros use 5V, on 3.3V devices use 3.3V)
SDA to the **SDA** (blue wire on **STEMMA QT** version) i2c data pin
SCL to the **SCL** (yellow wire on **STEMMA QT** version) i2c clock pin

There are internal 10K pull-ups on the PCF8523 on SDA and SCL to the VCC voltage



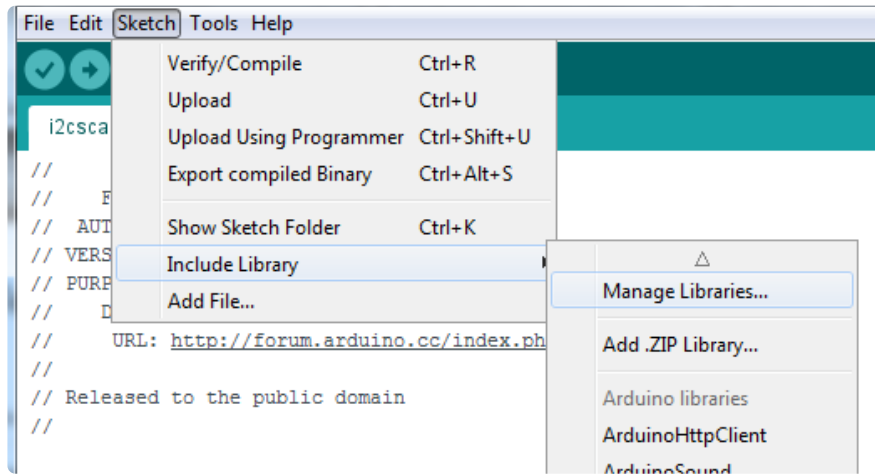
pcfmetro Fritzing

<https://adafru.it/A1F>

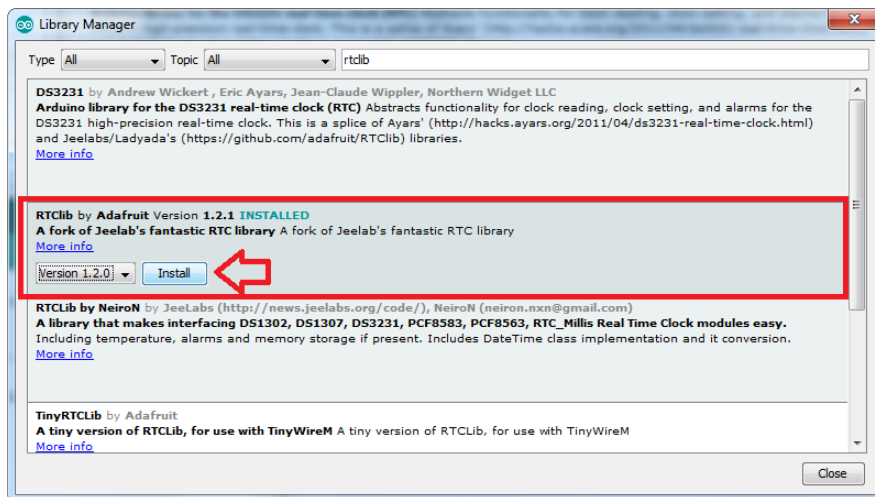
Talking to the RTC

The RTC is an i2c device, which means it uses 2 wires to to communicate. These two wires are used to set the time and retrieve it.

For the RTC library, we'll be using a fork of JeeLab's excellent RTC library, [which is available on GitHub \(https://adafru.it/c7r\)](https://adafru.it/c7r). You can do that by visiting the github repo and manually downloading or, easier go to the **Arduino Library Manager**



Type in **RTCLib** - and find the one that is by **Adafruit** and click **Install**



There are a few different 'forks' of RTCLib, make sure you are using the ADAFRUIT one!

We also have a great tutorial on Arduino library installation at:
<http://learn.adafruit.com/adafruit-all-about-arduino-libraries-install-use>

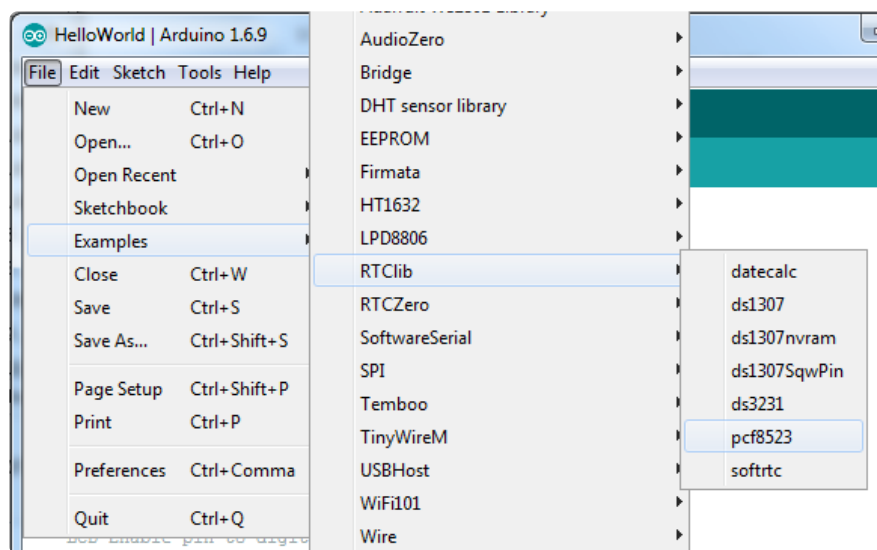
Once done, restart the IDE

First RTC test

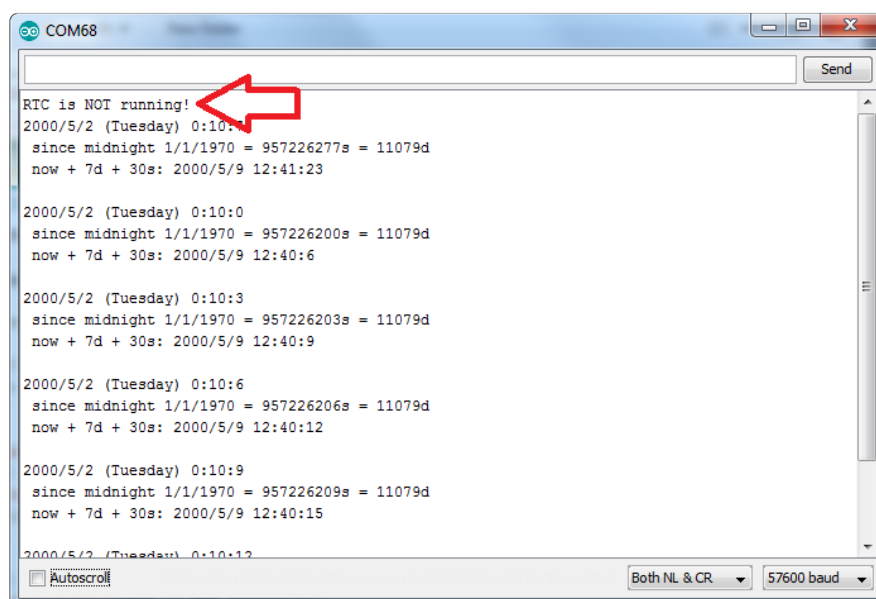
The first thing we'll demonstrate is a test sketch that will read the time from the RTC once a second. We'll also show what happens if you remove the battery and replace it since that causes the RTC to halt. So to start, remove the battery from the holder while the Feather is not powered or plugged into USB. Wait 3 seconds and then replace the battery. This resets the RTC chip. Now load up the matching sketch for your RTC

Open up **Examples->RTCLib->pcf8523**

Upload it to your board with the PCF8523 breakout board or FeatherWing connected



Now open up the Serial Console and make sure the baud rate is set correctly at **57600 baud** you should see the following:



Whenever the RTC chip loses all power (including the backup battery) it will reset to an earlier date and report the time as 0:0:0 or similar. Whenever you set the time, this will kickstart the clock ticking.

So, basically, the upshot here is that you should never ever remove the battery once you've set the time. You shouldn't have to and the battery holder is very snug so unless the board is crushed, the battery won't 'fall out'

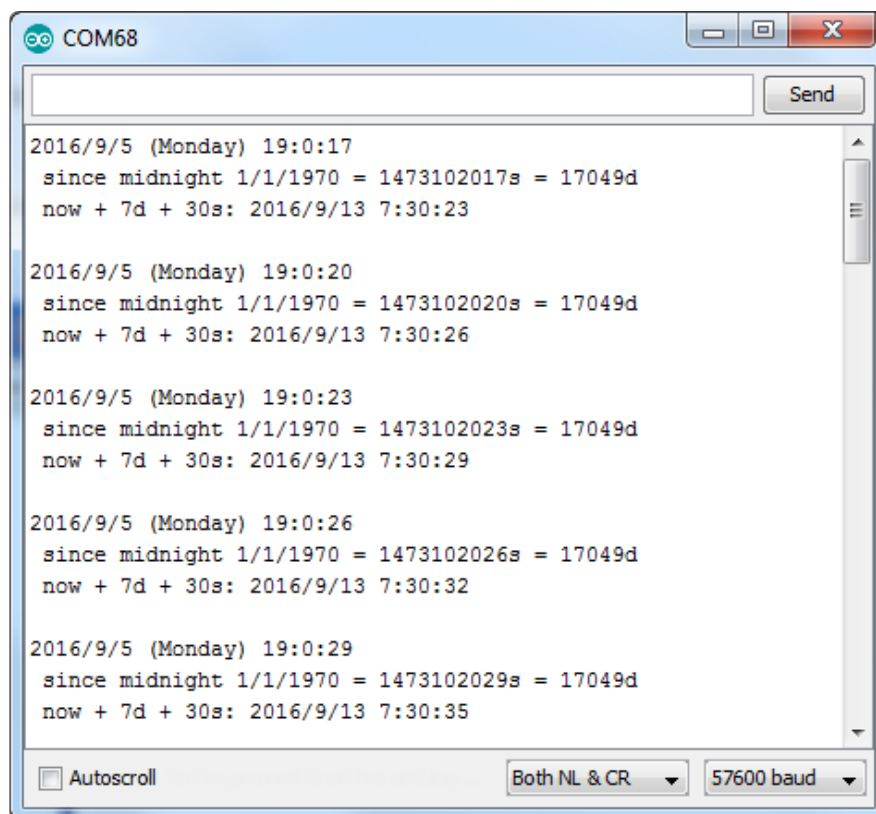
Setting the time

With the same sketch loaded, uncomment the line that starts with **RTC.adjust** like so:

```
if (! rtc.initialized()) {  
  Serial.println("RTC is NOT running!");  
  // following line sets the RTC to the date & time this sketch was compiled  
  rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));  
}
```

This line is very cute, what it does is take the Date and Time according the computer you're using (right when you compile the code) and uses that to program the RTC. If your computer time is not set right you should fix that first. Then you must press the **Upload** button to compile and then immediately upload. If you compile and then upload later, the clock will be off by that amount of time.

Then open up the Serial monitor window to show that the time has been set



From now on, you won't have to ever set the time again: the battery will last 5 or more years

Reading the time

Now that the RTC is merrily ticking away, we'll want to query it for the time. Let's look at the sketch again to see how this is done

```
void loop () {
    DateTime now = rtc.now();

    Serial.print(now.year(), DEC);
    Serial.print('/');
    Serial.print(now.month(), DEC);
    Serial.print('/');
    Serial.print(now.day(), DEC);
    Serial.print(" ");
    Serial.print(daysOfTheWeek[now.dayOfTheWeek()]);
    Serial.print(" ");
    Serial.print(now.hour(), DEC);
    Serial.print(':');
    Serial.print(now.minute(), DEC);
    Serial.print(':');
    Serial.print(now.second(), DEC);
    Serial.println();
}
```

There's pretty much only one way to get the time using the RTCLib, which is to call **now()**, a function that returns a **DateTime** object that describes the year, month, day, hour, minute and second when you called **now()**.

There are some RTC libraries that instead have you call something like **RTC.year()** and **RTC.hour()** to get the current year and hour. However, there's one problem where if you happen to ask for the minute right at **3:14:59** just before the next minute rolls over, and then the second right after the minute rolls over (so at **3:15:00**) you'll see the time as **3:14:00** which is a minute off. If you did it the other way around you could get **3:15:59** - so one minute off in the other direction.

Because this is not an especially unlikely occurrence - particularly if you're querying the time pretty often - we take a 'snapshot' of the time from the RTC all at once and then we can pull it apart into **day()** or **second()** as seen above. It's a tiny bit more effort but we think it's worth it to avoid mistakes!

We can also get a 'timestamp' out of the **DateTime** object by calling **unixtime** which counts the number of seconds (not counting leapseconds) since midnight, January 1st 1970

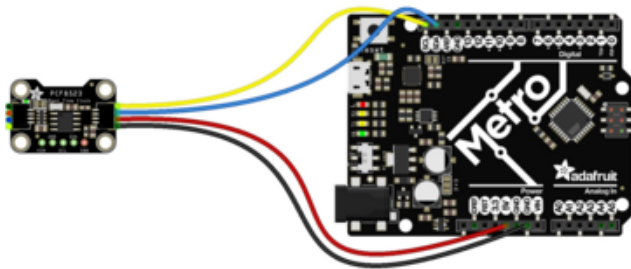
```
Serial.print(" since 2000 = ");
Serial.print(now.unixtime());
Serial.print("s = ");
Serial.print(now.unixtime() / 86400L);
Serial.println("d");
```

Since there are $60*60*24 = 86400$ seconds in a day, we can easily count days since then as well. This might be useful when you want to keep track of how much time has passed since the last query, making some math a lot easier (like checking if it's been 5 minutes later, just see if `unixtime()` has increased by 300, you dont have to worry about hour changes)

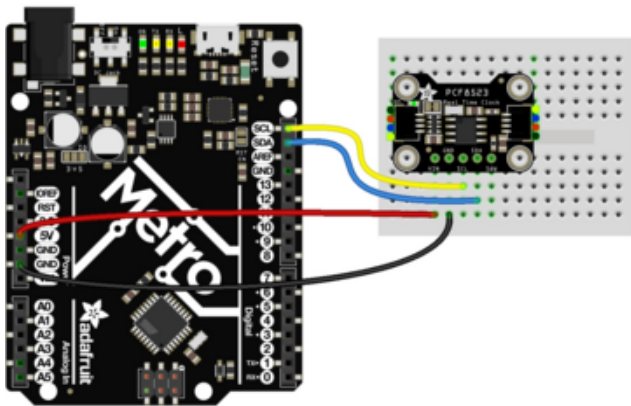
RTC with CircuitPython

Wiring

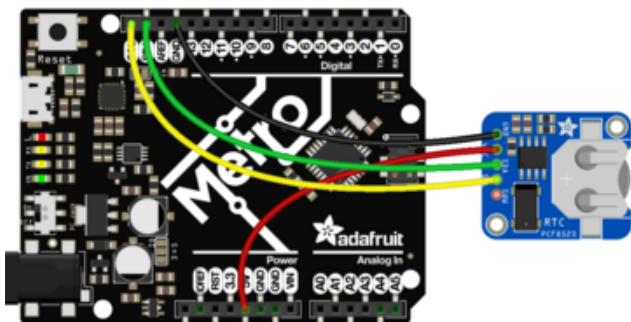
Wiring it up is easy, connect it up as shown below.



fritzing



fritzing



fritzing

GND to GND on your board
VCC to the logic level power of your board
 - every CircuitPython board uses 3.3V
SDA to the **SDA** i2c data pin
SCL to the **SCL** i2c clock pin

There are internal 10K pull-ups on the PCF8523 on SDA and SCL to the VCC voltage

Adafruit CircuitPython Library Install

To use the RTC sensor with your [Adafruit CircuitPython](https://adafru.it/AIP) (<https://adafru.it/AIP>) board you'll need to install the [Adafruit_CircuitPython_PCF8523](https://adafru.it/Bvh) (<https://adafru.it/Bvh>) module on your board.

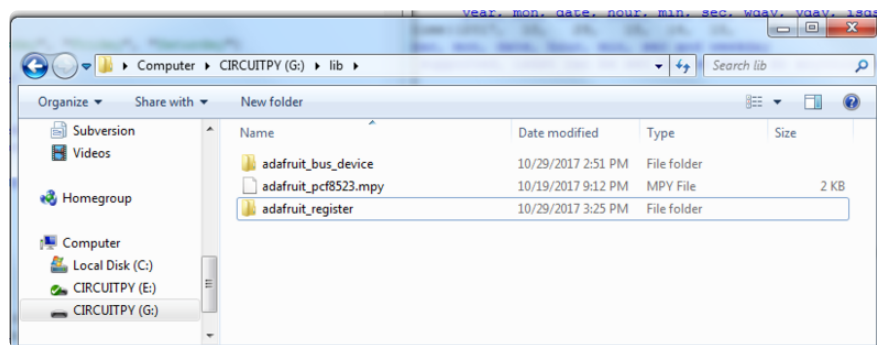
First make sure you are running the [latest version of Adafruit CircuitPython](https://adafru.it/Amd) (<https://adafru.it/Amd>) for your board.

Next you'll need to install the necessary libraries to use the hardware--carefully follow the steps to find and install these libraries from [Adafruit's CircuitPython library bundle](https://adafru.it/zdx) (<https://adafru.it/zdx>). Our introduction guide has [a great page on how to install the library bundle](https://adafru.it/ABU) (<https://adafru.it/ABU>) for both express and non-express boards.

Remember for non-express boards like the, you'll need to manually install the necessary libraries from the bundle:

- **adafruit_bus_device** folder
- **adafruit_register** folder
- **adafruit_pcf8523.mpy**

Before continuing make sure your board's **lib** folder or root filesystem has the **adafruit_pcf8523.mpy** module, the **adafruit_register** folder, and the **adafruit_bus_device** folder copied over.



Usage

To demonstrate the usage of the PCF8523 module you can connect to your board's serial REPL to see the output while saving our example sketch to **code.py**

Next [connect to the board's serial REPL](https://adafru.it/Awz) (<https://adafru.it/Awz>) so you are at the CircuitPython >>> prompt.

Then save this script to **code.py** (back up or remove whatever was there before)

```
# SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
# SPDX-License-Identifier: MIT

# Simple demo of reading and writing the time for the PCF8523 real-time clock.
# Change the if False to if True below to set the time, otherwise it will just
# print the current date and time every second. Notice also comments to adjust
# for working with hardware vs. software I2C.

import time

import board

from adafruit_pcf8523.pcf8523 import PCF8523
```

```

i2c = board.I2C() # uses board.SCL and board.SDA
# i2c = board.STEMMA_I2C() # For using the built-in STEMMMA QT connector on a
microcontroller
rtc = PCF8523(i2c)

# Lookup table for names of days (nicer printing).
days = ("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday",
"Sunday")

if False: # change to True if you want to set the time!
    # year, mon, date, hour, min, sec, wday, yday, isdst
    t = time.struct_time((2017, 10, 29, 10, 31, 0, 0, -1, -1))
    # you must set year, mon, date, hour, min, sec and weekday

# yearday is not supported, isdst can be set but we don't do anything with it at
this time
    print("Setting time to:", t) # uncomment for debugging
    rtc.datetime = t
    print()
# pylint: enable-msg=using-constant-test

# Main loop:
while True:
    t = rtc.datetime
    # print(t) # uncomment for debugging
    print(f"The date is {days[int(t.tm_wday)]} {t.tm_mday}/{t.tm_mon}/{t.tm_year}")
    print(f"The time is {t.tm_hour}:{t.tm_min:02}:{t.tm_sec:02}")
    time.sleep(1) # wait a second

```

Setting the time

The first time you run the program, you'll need to **set the time**

find these lines:

```

if False: # change to True if you want to set the time!
    # year, mon, date, hour, min, sec, wday, yday, isdst
    t = time.struct_time((2017, 10, 29, 10, 31, 0, 0, -1, -1))
    # you must set year, mon, date, hour, min, sec and weekday

# yearday is not supported, isdst can be set but we don't do anything with it at
this time
    print("Setting time to:", t) # uncomment for debugging
    rtc.datetime = t
    print()

```

Change the **False** to **True** in the first line, and update the `time.struct_time` to have the current time starting from `year` to `weekday`. The last two entries can stay at -1

Re-run the sketch by saving and you'll see this out of the REPL:

```
Adafruit CircuitPython REPL
main.py output:
Setting time to: struct_time(tm_year=2017, tm_mon=10, tm_mday=29,
tm_hour=15, tm_min=14, tm_sec=15, tm_wday=0, tm_yday=-1, tm_isdst=-1)

The date is Sunday 29/10/2017
The time is 15:14:15
The date is Sunday 29/10/2017
The time is 15:14:16
The date is Sunday 29/10/2017
The time is 15:14:17
The date is Sunday 29/10/2017
The time is 15:14:18
```

Note the part where the program says it is **Setting time to**:

Now you can go back and change the if **True** to if **False** and save, so you don't re-set the RTC again.

The script will now output the time and date

```
main.py output:
The date is Sunday 29/10/2017
The time is 15:14:56
The date is Sunday 29/10/2017
The time is 15:14:57
The date is Sunday 29/10/2017
The time is 15:14:58
```

Using the SD Card

The other half of the adalogger FeatherWing is the SD card. The SD card is how we store long term data. While the Feather may have a permanent EEPROM storage, its only a couple hundred bytes - tiny compared to a 2 gig SD card. SD cards are so cheap and easy to get, its an obvious choice for long term storage so we use them for the 'Wing!

The FeatherWing kit doesn't come with an SD card but [we carry one in the shop that is guaranteed to work](https://adafru.it/alH) (<https://adafru.it/alH>). Pretty much any SD card should work but be aware that some cheap cards are 'fakes' and can cause headaches.



4GB Blank SD/MicroSD Memory Card

Add mega-storage in a jiffy using this 4 GB micro-SD card. It comes with a SD adapter so you can use it with any of our shields or adapters! Preformatted to FAT so it works out of...

<https://www.adafruit.com/product/102>

You'll also need a way to read and write from the SD card. Sometimes you can use your camera and MP3 player - when its plugged in you will be able to see it as a disk. Or you may need an [SD card reader](http://adafru.it/939) (<http://adafru.it/939>). The Wing **doesnt** have the ability to display the SD card as a 'hard disk' like some MP3 players or games, the Feather does not have the hardware for that, so you will need an external reader!



USB MicroSD Card Reader/Writer - microSD / microSDHC / microSDXC

This is the cutest little microSD card reader/writer - but don't be fooled by its adorableness! It's wicked fast and supports up to 64 GB SDXC cards! Simply slide the card into...

<https://www.adafruit.com/product/939>

Formatting under Windows/Mac

If you bought an SD card, chances are it's already pre-formatted with a FAT filesystem. However you may have problems with how the factory formats the card, or if it's an old card it needs to be reformatted. The Arduino SD library we use supports both **FAT16** and **FAT32** filesystems.

If you have a very small SD card, say 8-32 Megabytes you might find it is formatted **FAT12** which isn't supported. You'll have to reformat these card. Either way, it's **always** good idea to format the card before using, even if it's new! Note that formatting will erase the card so save anything you want first.

Cards above 32GB cannot use FAT16 so try to keep your card use to 32GB and below.

We strongly recommend you use the official SD card formatter utility - written by the SD association it solves many problems that come with bad formatting!

The official SD formatter is available from https://www.sdcard.org/downloads/formatter_4/ (<https://adafru.it/cfL>)

Download it and run it on your computer, there's also a manual linked from that page for use

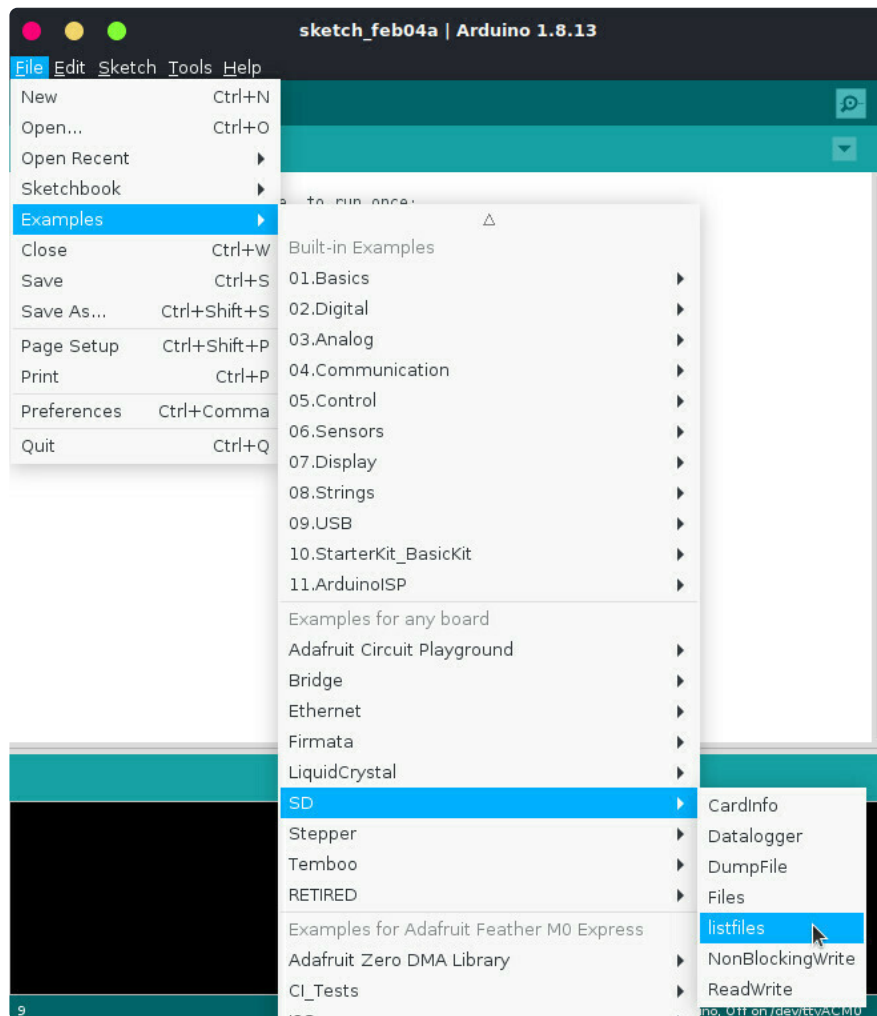
Download the official SD Formatter
software for Windows

<https://adafru.it/cfL>

Basic SD Card Test

The Arduino SD Card library has a built in example that will help you test the wiring and your connections

Open the file **listfiles** example sketch in the **SD** library:

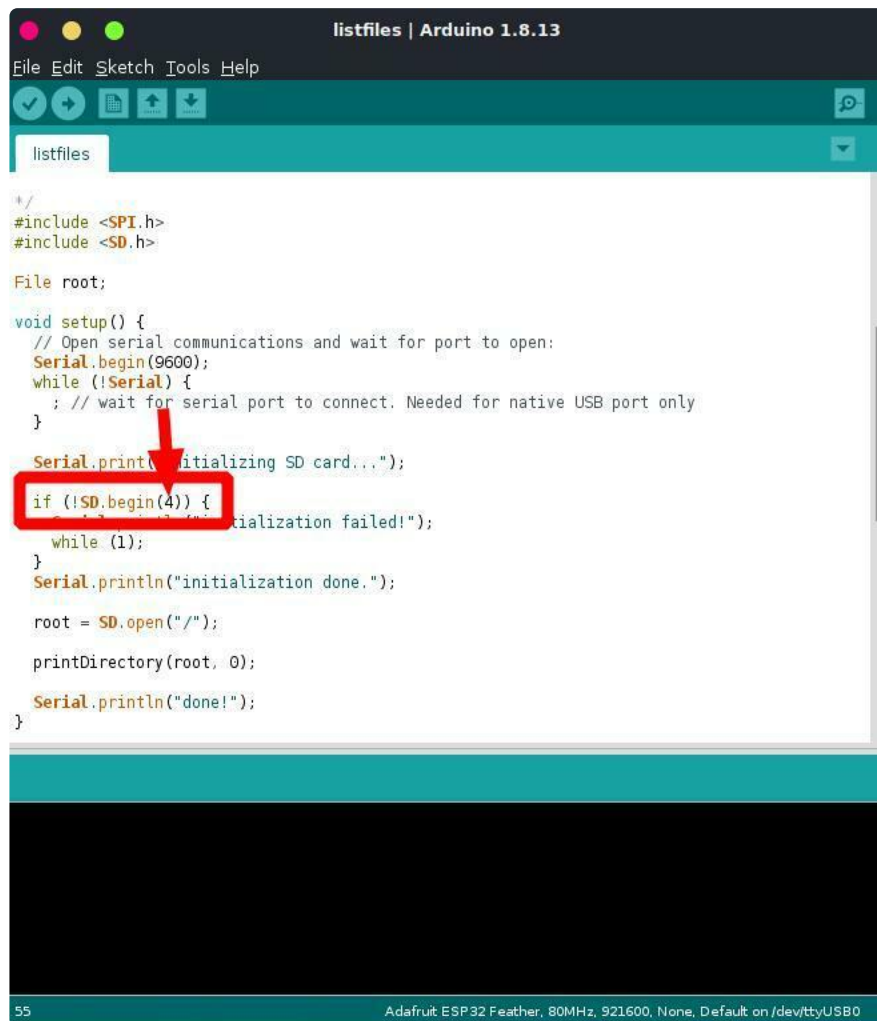


This sketch will not write any data to the card, just list the contents. This can be **very** useful when trying to figure out whether an SD card is supported. Before trying out a new card, please try out this sketch!

Scroll to the line where you see `SD.begin()` and change the value in the parentheses to match the chip select (CS) pin for your board.

- On ESP8266, the SD CS pin is on GPIO 15

- On Atmel M0, M4, 328p or 32u4 it's on GPIO 10
- On Teensy 3.x it's on GPIO 10
- On STM32F2/WICED, its on PB5
- On ESP32, it's on GPIO 33
- On nRF52832, it's on GPIO 11
- On nRF52840, it's on GPIO 10
- On ESP32-C6, its on GPIO 8



```
listfiles | Arduino 1.8.13
File Edit Sketch Tools Help

listfiles

*/
#include <SPI.h>
#include <SD.h>

File root;

void setup() {
  // Open serial communications and wait for port to open:
  Serial.begin(9600);
  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }

  Serial.print("Initializing SD card...");
  if (!SD.begin(4)) {
    // initialization failed!
    while (1);
  }
  Serial.println("initialization done.");

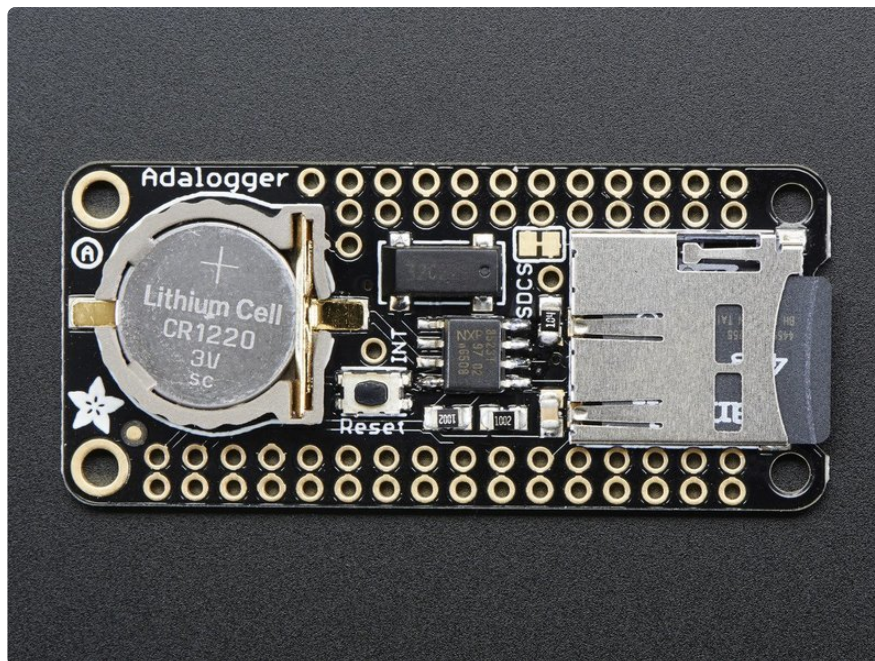
  root = SD.open("/");

  printDirectory(root, 0);

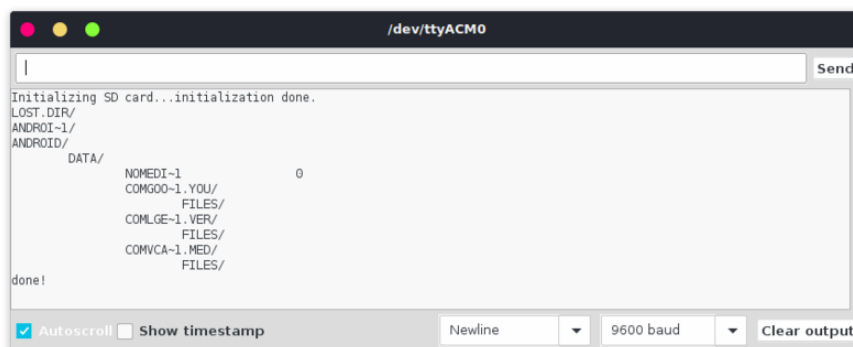
  Serial.println("done!");
}

55 Adafruit ESP32 Feather, 80MHz, 921600, None, Default on /dev/ttyUSB0
```

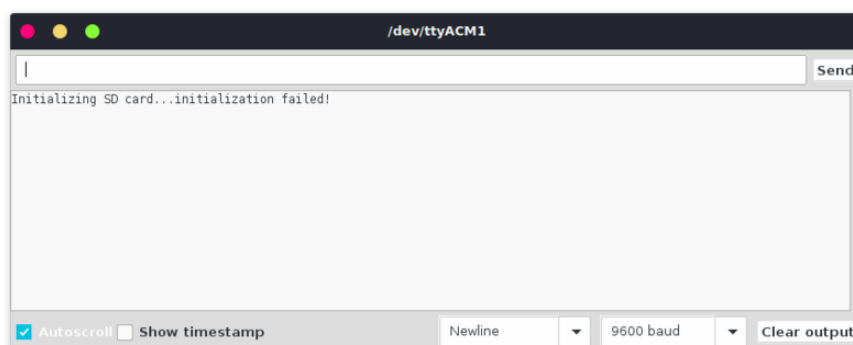
OK, now insert the micro SD card into the FeatherWing and upload the sketch.



Open up the **Serial Monitor** and you should see a listing of the files and folder layout. The specifics will depend on the card contents, but should look something like:



If you have a bad card, or some other formatting issue, you'll probably see:



It couldn't even initialize the SD card. This can also happen if there's a soldering error or if the card is really damaged

If you're having SD card problems, we suggest using the SD formatter mentioned above first to make sure the card is clean and ready to use!

Next steps!

Once you know the SD card works, check out the [SD card library](https://adafru.it/ucu) (<https://adafru.it/ucu>) examples, [SD library documentation](https://adafru.it/ucu) (<https://adafru.it/ucu>) and [Notes](https://adafru.it/ucv) (<https://adafru.it/ucv>)!

Note that the filesystem used is FAT which has 8 character file names, a period and a three character extension. So a valid filename could be file0823.log but not MyFile2024-02-29.data

Example logging sketch

If you want to try saving data to the SD card in the simplest sketch, try this example. You can adjust the **delay()** to set how often analog data is read from pin **A0** and saved to the SD card. The red LED will blink if there's an error, and the green LED will blink when data is written to the SD card.

```
// SPDX-FileCopyrightText: 2019 Limor Fried/ladyada for Adafruit Industries
//
// SPDX-License-Identifier: MIT

// Adalogger FeatherWing data logger, log data on pin A0

#include <SPI.h>
#include <SD.h>

// Set the pins used, varies depending on the Feather
#if defined(ESP8266)
  #define LED_RED 0
  #define SD_CS 15
#elif defined(ARDUINO_ADAFRUIT_FEATHER_ESP32C6)
  #define LED_RED LED_BUILTIN
  #define SD_CS 8
#elif defined(ESP32)
  #define LED_RED 13
  #define SD_CS 33
#elif defined(ARDUINO_STM32F2_FEATHER)
  #define LED_RED PB4
  #define SD_CS PB5
#elif defined(TEENSYDUINO)
  #define LED_RED 13
  #define SD_CS 10
#elif defined(ARDUINO_FEATHER52832)
  #define LED_RED 17
  #define SD_CS 11
#else // 32u4, M0 or 328
  #define LED_RED LED_BUILTIN
  #define SD_CS 4
#endif

File logfile;

// blink out an error code
void error(uint8_t errnum) {
  while(1) {
```

```

    uint8_t i;
    for (i=0; i<errnum; i++) {
        digitalWrite(LED_RED, HIGH);
        delay(100);
        digitalWrite(LED_RED, LOW);
        delay(100);
        yield();
    }
    for (i=errnum; i<10; i++) {
        delay(200);
        yield();
    }
}

void setup() {
    Serial.begin(115200);

    Serial.println("\r\nAnalog logger test");
    pinMode(LED_RED, OUTPUT);

    // see if the card is present and can be initialized:
    if (!SD.begin(SD_CS)) {
        Serial.println("Card init. failed!");
        error(2);
    }

    Serial.println("SD card OK");
    File root = SD.open("/");
    printDirectory(root, 0);

    char filename[15];
    strcpy(filename, "/ANALOG00.TXT");
    for (uint8_t i = 0; i < 100; i++) {
        filename[7] = '0' + i/10;
        filename[8] = '0' + i%10;
        // create if does not exist, do not open existing, write, sync after write
        if (!SD.exists(filename)) {
            break;
        }
    }

    logfile = SD.open(filename, FILE_WRITE);
    if( ! logfile ) {
        Serial.print("Couldnt create ");
        Serial.println(filename);
        error(3);
    }
    Serial.print("Writing to ");
    Serial.println(filename);
    Serial.println("Ready!");
}

void loop() {
    digitalWrite(LED_RED, HIGH); // LED onn during logging
    // Read the data on pin A0 and log it to SD card and to serial
    logfile.print("A0 = "); logfile.println(analogRead(0));
    Serial.print("A0 = "); Serial.println(analogRead(0));
    digitalWrite(LED_RED, LOW); // LED off when not actively logging

    // save the output!
    logfile.flush();

    delay(100); // Change the delay to space out readings
}

void printDirectory(File dir, int numTabs) {
    while(true) {

```

```

File entry = dir.openNextFile();
if (! entry) {
  // no more files
  break;
}
for (uint8_t i=0; i<numTabs; i++) {
  Serial.print('\t');
}
Serial.print(entry.name());
if (entry.isDirectory()) {
  Serial.println("/");
  printDirectory(entry, numTabs+1);
} else {
  // files have sizes, directories do not
  Serial.print("\t\t");
  Serial.println(entry.size(), DEC);
}
entry.close();
}
}

```

You will need to change the sketch's SD_CS pin to match the SD card's Chip Select pin on your Feather if it is not detected by the code at the beginning of the sketch.

If you really want to make sure you save every data point, ensure there is a

```
logfile.flush();
```

right after the **logfile.print**'s. The example does this.

However this will cause the adalogger to draw a lot more power, maybe about 3x as much on average (30mA avg rather than about 10mA).

CircuitPython

As of CircuitPython 9, a mount point (folder) named **/sd** is required on the **CIRCUITPY** drive. Make sure to create that directory after upgrading CircuitPython.

Follow these steps to create the /sd directory

<https://adafru.it/19ei>

Adafruit CircuitPython Module Install

To use a microSD card with your Adafruit CircuitPython board you'll need to install the [Adafruit_CircuitPython_SD](https://adafru.it/zwC) (<https://adafru.it/zwC>) module on your board.

First make sure you are running the [latest version of Adafruit CircuitPython](https://adafru.it/Amd) (<https://adafru.it/Amd>) for your board.

Next you'll need to install the necessary libraries to use the hardware--carefully follow the steps to find and install these libraries from [Adafruit's CircuitPython library bundle](https://adafru.it/zdx) (<https://adafru.it/zdx>). Our introduction guide has [a great page on how to install the library bundle](https://adafru.it/ABU) (<https://adafru.it/ABU>) for both express and non-express boards. **Be sure to use the latest CircuitPython Bundle** as it includes an updated version of the SD card module with a few necessary fixes!

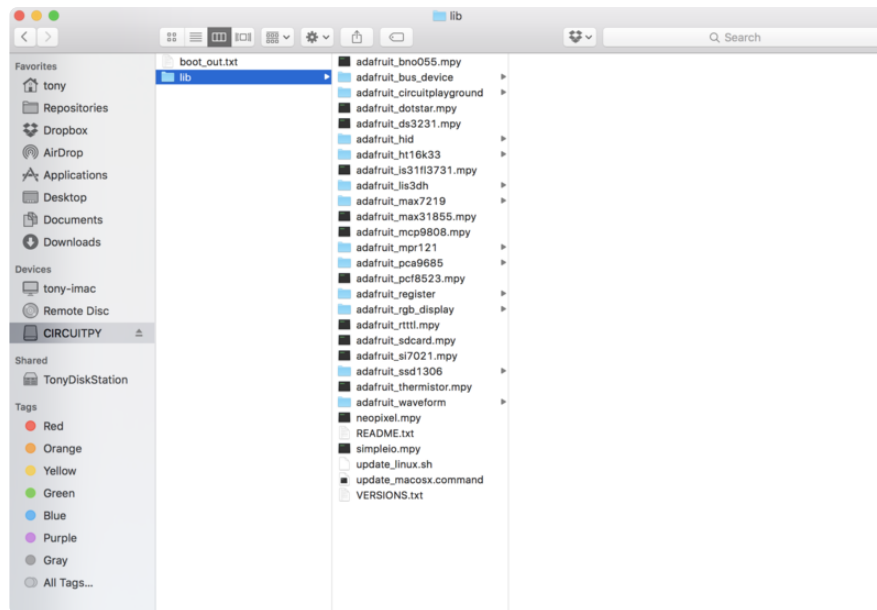
[If your board supports sdcardio](https://adafru.it/-Cy) (<https://adafru.it/-Cy>), then this is the preferred method to do things. **sdcardio** is a built-in module on boards that support it, so you don't have to copy it over.

Remember for non-express boards like the Feather M0 Basic, you'll need to manually install the necessary libraries from the bundle:

- **adafruit_sdcard.mpy**
- **adafruit_bus_device**

If your board doesn't support USB mass storage, like the ESP8266, then [use a tool like ampy to copy the file to the board](https://adafru.it/s1f) (<https://adafru.it/s1f>). You can use the latest version of ampy and its [new directory copy command](https://adafru.it/q2A) (<https://adafru.it/q2A>) to easily move module directories to the board.

Before continuing make sure your board's lib folder or root filesystem has the **adafruit_sdcard.mpy** and **adafruit_bus_device** modules copied over.



Usage

The following section will show how to initialize the SD card and read & write data to it from the board's Python prompt / REPL.

Next [connect to the board's serial REPL \(https://adafru.it/Awz\)](https://adafru.it/Awz) so you are at the CircuitPython >>> prompt.

sdcardio is an SPI interface SD card library in CircuitPython 6.0 that is optimized in C to be much faster than the original `adafruit_sdcard` library. Some boards don't have enough memory for this module, in which case you will have to use `adafruit_sdcard`

Initialize & Mount SD Card Filesystem Using **sdcardio**

Before using this method, verify that your board supports `sdcardio` using the [support matrix \(https://adafru.it/-Cy\)](https://adafru.it/-Cy). If it does not, try using `adafruit_sdcard` instead.

Before you can use the microSD card you need to initialize its SPI connection and mount its filesystem. First import all the modules we'll need:

```
import board
import busio
import sdcardio
import storage
```

Next create the SPI bus and a digital output for the microSD card's chip select line (be sure to select the right pin names for your wiring):

```
# Use the board's primary SPI bus
spi = board.SPI()
# Or, use an SPI bus on specific pins:
#spi = busio.SPI(board.SD_SCK, MOSI=board.SD_MOSI, MISO=board.SD_MISO)

# For breakout boards, you can choose any GPIO pin that's convenient:
cs = board.D10
# Boards with built in SPI SD card slots will generally have a
# pin called SD_CS:
#cs = board.SD_CS
```

If you are using some non-standard CircuitPython board, you may need to use specific GPIO pins for MISO and MOSI to get the SPI recognized appropriately

If your SPI chip select pin is different or not defined like SD_CS, put the appropriate pin number in the code, like board.D5.

Note that when you use `sdcardio`, `cs` is a `Pin` object, not a `DigitalInOut` object. If you change your code to use `adafruit_sdcard`, you need to use a `DigitalInOut` object instead.

At this point you're ready to create the microSD card object and the filesystem object:

```
sdcard = sdcardio.SDCard(spi, cs)
vfs = storage.VfsFat(sdcard)
```

Notice the `sdcardio` module has a `SDCard` class which contains all the logic for talking to the microSD card at a low level. This class needs to be told the SPI bus and chip select pin in its constructor.

After a `SDCard` instance is created it can be passed to the `storage` module's `VfsFat` class. This class has all the logic for translating CircuitPython filesystem calls into low level microSD card access. Both the `SDCard` and `VfsFat` class instances are required to mount the card as a new filesystem.

Finally you can mount the microSD card's filesystem into the CircuitPython filesystem. For example to make the path `/sd` on the CircuitPython filesystem read and write from the card run this command:

```
storage.mount(vfs, "/sd")
```

At this point, you can read and write to the SD card using common Python functions like `open`, `read`, and `write`. The filenames will all begin with `"/sd/"` to differentiate them from the files on the **CIRCUITPY** drive. If you're not familiar, and all this worked, skip ahead to the section labeled **Reading & Writing Data**.

If the same SPI bus is shared with other peripherals, it is important that the SD card be initialized before accessing any other peripheral on the bus. Failure to do so can prevent the SD card from being recognized until it is powered off or re-inserted.

Initialize & Mount SD Card Filesystem Using `adafruit_sdcard`

Before you can use the microSD card you need to initialize its SPI connection and mount its filesystem. First import the necessary modules to initialize the SPI and CS line physical connections:

```
import board
import busio
import digitalio
```

Next create the SPI bus and a digital output for the microSD card's chip select line (be sure to select the right pin name or number for your wiring):

```
spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)
# Use board.SD_CS for Feather M0 Adalogger
cs = digitalio.DigitalInOut(board.SD_CS)
# Or use a digitalio pin like 5 for breakout wiring:
#cs = digitalio.DigitalInOut(board.D5)
```

Now import modules to access the SD card and filesystem:

```
import adafruit_sdcard
import storage
```

At this point you're ready to create the microSD card object and the filesystem object:

```
sdcard = adafruit_sdcard.SDCard(spi, cs)
vfs = storage.VfsFat(sdcard)
```

Notice the `adafruit_sdcard` module has a **SDCard** class which contains all the logic for talking to the microSD card at a low level. This class needs to be told the SPI bus and chip select digital IO pin in its initializer.

After a **SDCard** class is created it can be passed to the **storage** module's **VfsFat** class. This class has all the logic for translating CircuitPython filesystem calls into low level microSD card access. Both the **SDCard** and **VfsFat** class instances are required to mount the card as a new filesystem.

Finally you can mount the microSD card's filesystem into the CircuitPython filesystem. For example to make the path `/sd` on the CircuitPython filesystem read and write from the card run this command:

```
storage.mount(vfs, "/sd")
```

The first parameter to the `storage.mount` command is the **VfsFat** class instance that was created above, and the second parameter is the location within the CircuitPython filesystem that you'd like to 'place' the microSD card. Remember the mount location as you'll need it to read and write files on the card!

If the same SPI bus is shared with other peripherals, it is important that the SD card be initialized before accessing any other peripheral on the bus. Failure to do so can prevent the SD card from being recognized until it is powered off or re-inserted.

Reading & Writing Data

Once the microSD card is mounted inside CircuitPython's filesystem you're ready to read and write data from it. Reading and writing data is simple using Python's file operations like [open](https://adafru.it/reL) (<https://adafru.it/reL>), [close](https://adafru.it/ryE) (<https://adafru.it/ryE>), [read](https://adafru.it/ryE) (<https://adafru.it/ryE>), and [write](https://adafru.it/ryE) (<https://adafru.it/ryE>). The beauty of CircuitPython and MicroPython is that they try to be as similar to desktop Python as possible, including access to files.

For example to create a file and write a line of text to it you can run:

```
with open("/sd/test.txt", "w") as f:  
    f.write("Hello world!\r\n")
```

Notice the `with` statement is used to create a context manager that opens and automatically closes the file. This is handy because with file access you Python you must close the file when you're done or else all the data you thought was written might be lost!

The `open` function is used to open the file by telling it the path to it, and the mode (`w` for writing). Notice the path is under `/sd`, `/sd/test.txt`. This means the file will be created on the microSD card that was mounted as that path.

Starting with CircuitPython 9, the SD card **MUST** be mapped to `/sd` on the CircuitPython file system. Create an empty `sd` directory in the root directory of the CIRCUITPY drive, especially if you get messages complaining of a missing mount point.

Inside the context manager you can access the `f` variable to operate on the file while it's open. The **write** function is called to write a line of text to the file. Notice that unlike a print statement you need to end the string passed to write with explicit carriage returns and new lines.

You can also open a file and read a line from it with similar code:

```
with open("/sd/test.txt", "r") as f:
    print("Read line from file:")
    print(f.readline())
```

If you wanted to read and print all of the lines from a file you could call **readline** in a loop. Once **readline** reaches the end of the file it will return an empty string so you know when to stop:

```
with open("/sd/test.txt", "r") as f:
    print("Printing lines in file:")
    line = f.readline()
    while line != '':
        print(line)
        line = f.readline()
```

There's even a **readlines** function that will read all of the lines in the file and return them in an array of lines. Be careful though as this means the entire file must be loaded into memory, so if the file is very large you might run out of memory. If you know your file is very small you can use it though:

```
with open("/sd/test.txt", "r") as f:
    lines = f.readlines()
    print("Printing lines in file:")
    for line in lines:
        print(line)
```

Finally one other very common file scenario is opening a file to add new data at the end, or append data. This works exactly the same as in Python and the **open** function can be told you'd like to append instead of erase and write new data (what normally happens with the **w** option for **open**). For example to add a line to the file:

```
with open("/sd/test.txt", "a") as f:
    f.write("This is another line!\r\n")
```

Notice the **a** option in the open function--this tells Python to add data at the end of the file instead of erasing it and starting over at the top. Try reading the file with the code above to see the new line that was added!

That's all there is to manipulating files on microSD cards with CircuitPython!

Here are a few more complete examples of using a SD card from the [Trinket M0 CircuitPython guides \(https://adafru.it/Bvi\)](https://adafru.it/Bvi). These are great as a reference for more SD card usage.

List Files

Load this into **code.py** (CircuitPython) or **main.py** (MicroPython):

```
# SPDX-FileCopyrightText: 2017 Limor Fried for Adafruit Industries
#
# SPDX-License-Identifier: MIT

import os

import adafruit_sdcard
import board
import busio
import digitalio
import storage

# Use any pin that is not taken by SPI
SD_CS = board.D0

# Connect to the card and mount the filesystem.
spi = busio.SPI(board.SCK, board.MOSI, board.MISO)
cs = digitalio.DigitalInOut(SD_CS)
sdcard = adafruit_sdcard.SDCard(spi, cs)
vfs = storage.VfsFat(sdcard)
storage.mount(vfs, "/sd")

# Use the filesystem as normal! Our files are under /sd

# This helper function will print the contents of the SD

def print_directory(path, tabs=0):
    for file in os.listdir(path):
        if file == "?":
            continue # Issue noted in Learn
        stats = os.stat(path + "/" + file)
        filesize = stats[6]
        isdir = stats[0] & 0x4000

        if filesize < 1000:
            sizestr = str(filesize) + " by"
        elif filesize < 1000000:
            sizestr = "%0.1f KB" % (filesize / 1000)
        else:
            sizestr = "%0.1f MB" % (filesize / 1000000)

        prettyprintname = ""
        for _ in range(tabs):
            prettyprintname += "    "
        prettyprintname += file
        if isdir:
```

```

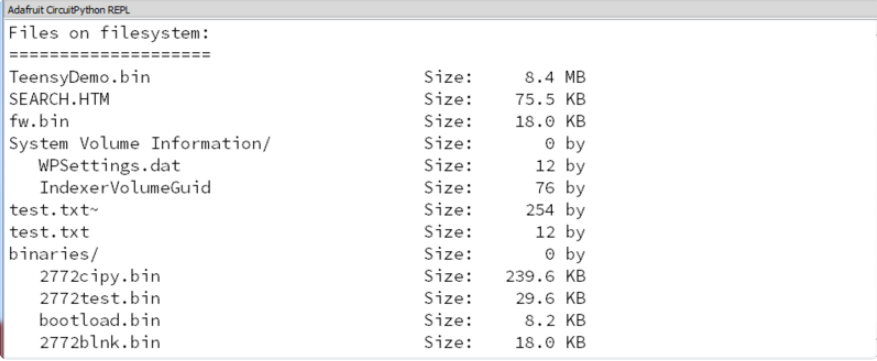
        prettyprintname += "/"
    print('{0:<40} Size: {1:>10}'.format(prettyprintname, sizestr))

    # recursively print directory contents
    if isdir:
        print_directory(path + "/" + file, tabs + 1)

print("Files on filesystem:")
print("=====")
print_directory("/sd")

```

Once it's loaded up, open up the REPL (and restart it with ^D if necessary) to get a printout of all the files included. We recursively print out all files and also the filesize. This is a good demo to start with because you can at least tell if your files exist!



```

Adafruit CircuitPython REPL
Files on filesystem:
=====
TeensyDemo.bin                Size:      8.4 MB
SEARCH.HTM                    Size:     75.5 KB
fw.bin                         Size:     18.0 KB
System Volume Information/
  WPSettings.dat               Size:      12 by
  IndexerVolumeGuid            Size:      76 by
test.txt~                      Size:     254 by
test.txt                       Size:      12 by
binaries/                      Size:       0 by
  2772copy.bin                 Size:    239.6 KB
  2772test.bin                 Size:     29.6 KB
  bootload.bin                 Size:      8.2 KB
  2772blnk.bin                 Size:     18.0 KB

```

Logging Temperature

But you probably want to do a little more, lets log the temperature from the chip to a file.

Here's the new script

```

# SPDX-FileCopyrightText: 2017 Limor Fried for Adafruit Industries
#
# SPDX-License-Identifier: MIT

import time

import adafruit_sdcard
import board
import busio
import digitalio
import microcontroller
import storage

# Use any pin that is not taken by SPI
SD_CS = board.D0

led = digitalio.DigitalInOut(board.D13)
led.direction = digitalio.Direction.OUTPUT

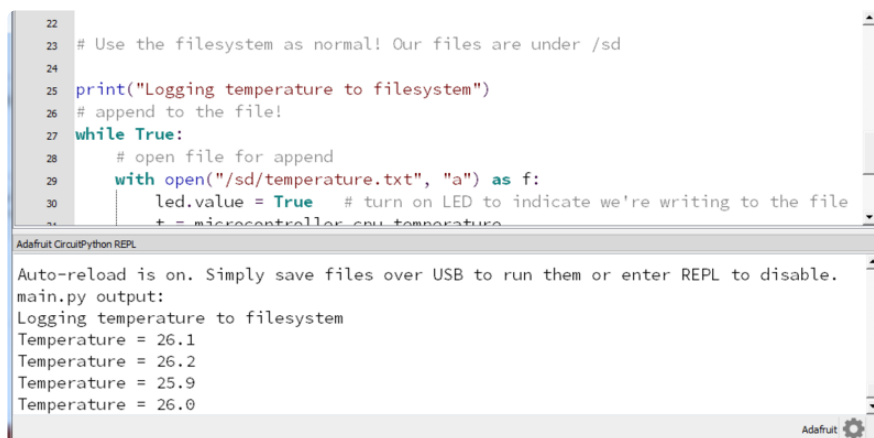
# Connect to the card and mount the filesystem.
spi = busio.SPI(board.SCK, board.MOSI, board.MISO)
cs = digitalio.DigitalInOut(SD_CS)
sdcard = adafruit_sdcard.SDCard(spi, cs)
vfs = storage.VfsFat(sdcard)
storage.mount(vfs, "/sd")

```

```
# Use the filesystem as normal! Our files are under /sd

print("Logging temperature to filesystem")
# append to the file!
while True:
    # open file for append
    with open("/sd/temperature.txt", "a") as f:
        led.value = True # turn on LED to indicate we're writing to the file
        t = microcontroller.cpu.temperature
        print("Temperature = %0.1f" % t)
        f.write("%0.1f\n" % t)
        led.value = False # turn off LED to indicate we're done
    # file is saved
    time.sleep(1)
```

When saved, the Trinket will start saving the temperature once per second to the SD card under the file **temperature.txt**



The key part of this demo is in these lines:

```
print("Logging temperature to filesystem")
# append to the file!
while True:
    # open file for append
    with open("/sd/temperature.txt", "a") as f:
        led.value = True # turn on LED to indicate we're writing to the file
        t = microcontroller.cpu.temperature
        print("Temperature = %0.1f" % t)
        f.write("%0.1f\n" % t)
        led.value = False # turn off LED to indicate we're done
    # file is saved
    time.sleep(1)
```

This is a slightly complex demo but it's for a good reason. We use **with** (a 'context') to open the file for **appending**, that way the file is only opened for the very short time its written to. This is safer because then if the SD card is removed or the board turned off, all the data will be safe(r).

The LED is used to let the person using this know that the temperature is being written, it turns on just before the write and then off right after.

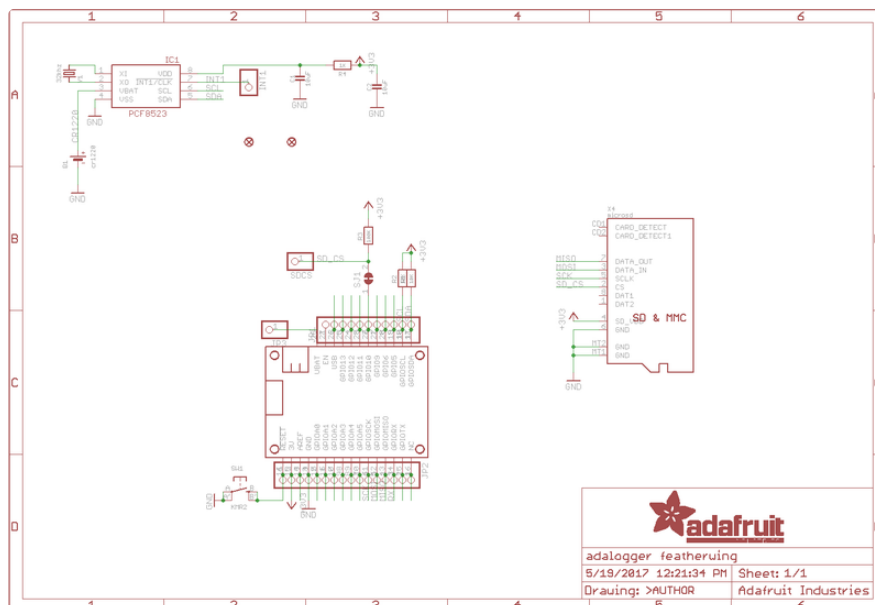
After the LED is turned off the **with** ends and the context closes, the file is safely stored.

Downloads

Datasheets and Files

- [EagleCAD PCB files on GitHub \(https://adafru.it/rek\)](https://adafru.it/rek)
- [3D models on GitHub \(https://adafru.it/11A4\)](https://adafru.it/11A4)
- [Fritzing object in Adafruit Fritzing library \(https://adafru.it/c7M\)](https://adafru.it/c7M)
- [PCF8523 product page \(https://adafru.it/reb\)](https://adafru.it/reb)

Schematic



Fabrication Print

